CAHIER DES CHARGES FONCTIONNEL ET TECHNIQUE

Projet : RisingTown

Simulation de Smart City Gamifiée et Centralisée

Équipe de Développement :
42014913 : Mohamed Boughmadi
42014728 : Mohamet Thiam
42004173 : Djama Faiz

Version	1.0 (Finale)
Date	30 mars 2026
Environnement	Windows / Python Django
URL du Site	`https://boughbough.pythonanywhere.com`

Document confidentiel destiné à l'évaluation du projet.

Table des matières

1 Contexte et Justification du Projet **3**
1.1 Introduction . . . 3
1.1.1 Organigramme du Projet . . . 3
1.2 Analyse de l'Existant et Problématique . . . 4
1.3 La Solution RisingTown . . . 4
1.3.1 Comparatif : Monde Réel vs RisingTown . . . 4
1.3.2 Intégration dans l'Écosystème MIAGE (Partenariat RisingFoot) . . . 4
1.4 Objectifs . . . 5
1.5 Méthodologie de Gestion de Projet . . . 5
1.5.1 Organisation de l'Équipe (Rôles Scrum) . . . 5
1.5.2 Planification des Sprints . . . 5
1.6 Maquettes de l'Interface Utilisateur (UI) . . . 6
1.6.1 Authentification et Accueil . . . 6
1.6.2 Espace Personnel du Citoyen . . . 7
1.6.3 Immersion et Cycle Temporel . . . 7
1.6.4 Outils de Communication et Justice . . . 8

2 Analyse et Spécification des Besoins **11**
2.1 Les Acteurs du Système . . . 11
2.2 Formalisation des Besoins . . . 11
2.2.1 Besoins Fonctionnels (Ce que le système doit faire) . . . 11
2.2.2 Besoins Non Fonctionnels (Qualité de service) . . . 12
2.3 Modélisation des Interactions (Use Case) . . . 13
2.4 Règles de Gestion (Business Rules) . . . 14
2.4.1 Algorithme Fiscal (Impôts Progressifs) . . . 14
2.4.2 Système Judiciaire et Carcéral . . . 14
2.4.3 Cycle Économique (Salaires et Loyers) . . . 14
2.5 Spécifications Fonctionnelles par Rôle . . . 14
2.5.1 Espace Citoyen (Utilisateur Standard) . . . 14
2.5.2 Espace Directeur (Gestionnaire) . . . 15
2.5.3 Espace Adjoint (Administration) . . . 15
2.5.4 Espace Maire (Super-Administrateur) . . . 15
2.6 Product Backlog (Liste des User Stories) . . . 16

3 Conception Technique **18**
3.1 Environnement de Développement . . . 18
3.2 Justification des Choix Techniques . . . 18
3.2.1 Comparatif Framework Backend : Django vs Flask . . . 18

3.2.2 Architecture MVT (Modèle-Vue-Template) 18
3.3 Diagramme de Classes (Modèle de Données) 19
3.4 Dictionnaire des Données 21
3.4.1 Table : Citoyen (Le Joueur) 21
3.4.2 Table : Batiment (Infrastructures) 21
3.4.3 Table : Candidature (Système de Notifications) 21
3.4.4 Table : Transaction (Historique Bancaire) 22
3.5 Détail des Processus Critiques (Diagrammes de Séquence) 22
3.5.1 Analyse Approfondie du Processus de Clôture de Journée 22

4 Organisation et Livrables **25**
4.1 Livrables Fournis 25
4.2 Conclusion et Perspectives 25

Chapitre 1

Contexte et Justification du Projet

1.1 Introduction

Le projet **RisingTown** est une application web de simulation urbaine ("Smart City"). Elle permet à plusieurs utilisateurs d'interagir simultanément au sein d'une ville virtuelle persistante, en incarnant des rôles variés allant du simple citoyen au Maire, en passant par des directeurs d'infrastructures.

1.1.1 Organigramme du Projet

Le projet s'inscrit dans le cursus de Licence 3 MIAGE à l'Université Paris Nanterre. L'organisation hiérarchique et fonctionnelle est définie comme suit pour assurer le bon déroulement du développement Agile.

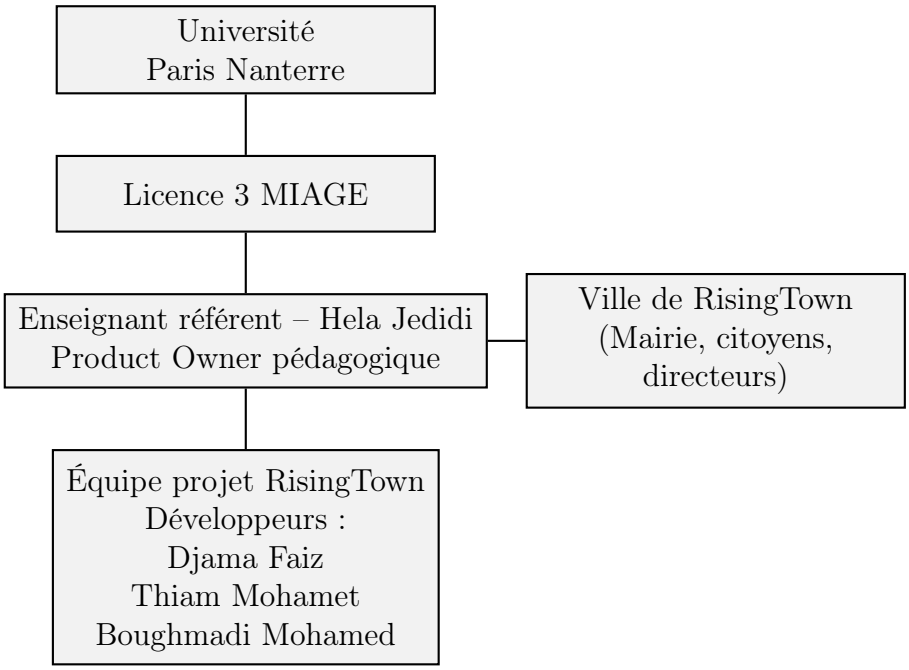


FIGURE 1.1 – Organigramme hiérarchique du projet RisingTown

Cette structure définit clairement les responsabilités :

— **Maîtrise d'Ouvrage Pédagogique (Product Owner) :** Mme **Hela Jedidi** (Enseignant référent) valide les fonctionnalités et les livrables.

- **Maîtrise d'Œuvre (Équipe de Développement) :**
  - **Djama Faiz** : Développeur, Gestionnaire BDD.
  - **Thiam Mohamet** : Développeur, Scrum Master.
  - **Boughmadi Mohamed** : Développeur, Architecte logiciel.

1.2 Analyse de l'Existant et Problématique

Dans le monde réel, l'interaction d'un citoyen avec sa ville et son administration est souvent complexe et fragmentée.

- **Fragmentation des interfaces :** Un citoyen doit utiliser un site pour ses impôts, un autre pour chercher un emploi, un troisième pour ses démarches de logement, et encore un autre pour contacter la mairie.
- **Manque de transparence :** L'impact des décisions politiques (taux d'imposition, budget municipal) sur la vie quotidienne est souvent obscur pour le grand public.
- **Absence de vision globale :** Il est difficile de visualiser comment l'économie, le social et l'urbanisme s'interconnectent.

1.3 La Solution RisingTown

RisingTown propose de **centraliser** toutes ces interactions au sein d'une interface unique et ludique.

1.3.1 Comparatif : Monde Réel vs RisingTown

Gestion Actuelle (Monde Réel)	Approche RisingTown
Multiplication des guichets et des sites web.	**Dashboard Unique :** Tout est accessible depuis une carte interactive et un tableau de bord.
Démarches administratives lentes (courrier, délais).	**Temps Réel :** Les candidatures, virements et plaintes sont traités instantanément ou par notifications.
Communication verticale (Mairie vers Citoyen).	**Communication Horizontale :** Système de "Téléphone" (Chat) intégré et publication d'actualités par les citoyens.
Complexité administrative.	**Gamification :** L'usage de jauges (Santé, Bonheur) et d'un score budgétaire rend la gestion intuitive.

1.3.2 Intégration dans l'Écosystème MIAGE (Partenariat RisingFoot)

Dans une démarche de cohérence globale avec les autres projets de la promotion L3 MIAGE, *RisingTown* intègre une dimension d'ouverture vers des services tiers.

Une collaboration a été mise en place avec le groupe en charge du projet de gestion sportive (Club de Football). Concrètement, cela se matérialise par :

— **Intégration Urbanistique :** La présence d'un bâtiment spécial "Stade" sur la carte de la ville.

— **Redirection Web :** Ce bâtiment agit comme une passerelle (Portail). Lorsqu'un citoyen interagit avec le stade, il est redirigé vers l'application web du Club de Foot gérée par l'autre groupe de projet.

— **Architecture Découplée :** Afin de maintenir l'indépendance des deux projets, il n'y a pas de jointure de base de données (pas de partage d'utilisateurs). L'intégration se fait par hyperlien, garantissant que la panne de l'un n'affecte pas l'autre (Couplage faible).

1.4 Objectifs

1. **Pédagogique :** Comprendre les flux économiques (Salaires ↔ Impôts ↔ Services Publics).
2. **Technique :** Développer une architecture robuste capable de gérer des états concurrents (Maire modifiant la ville pendant qu'un citoyen déménage).
3. **Social :** Favoriser l'interaction via un système de messagerie et de hiérarchie (Patron/Employé).

1.5 Méthodologie de Gestion de Projet

Pour mener à bien ce développement en équipe, nous avons adopté la méthode **Agile Scrum**. Ce cadre de travail itératif nous a permis de livrer des fonctionnalités opérationnelles régulièrement et de nous adapter aux contraintes techniques.

1.5.1 Organisation de l'Équipe (Rôles Scrum)

— **Product Owner :** Définit la vision du jeu (Règles économiques, lois) et priorise les fonctionnalités.

— **Scrum Master :** Facilite le travail technique (Configuration Django, Git) et lève les blocages.

— **Équipe de Développement :** Réalise l'implémentation, les tests et le design.

1.5.2 Planification des Sprints

Le projet a été découpé en cycles courts (Sprints) pour valider chaque étape :

— **Sprint 1 (Fondations) :** Configuration de l'environnement, Base de données, Authentification.

— **Sprint 2 (Vie Citoyenne) :** Gestion immobilière (Logement), Système d'Emploi et Économie de base.

— **Sprint 3 (Politique et Justice) :** Rôles Maire/Adjoint, Système d'Impôts, Prison et Police.

— **Sprint 4 (Communication) :** Chat temps réel (Téléphone), Notifications et Finitions.

1.6 Maquettes de l'Interface Utilisateur (UI)

Cette section présente les interfaces graphiques développées pour assurer une expérience utilisateur fluide et immersive.

1.6.1 Authentification et Accueil

L'accès à l'application est sécurisé. L'utilisateur commence par s'identifier, ce qui le mène à l'interface d'accueil principale.

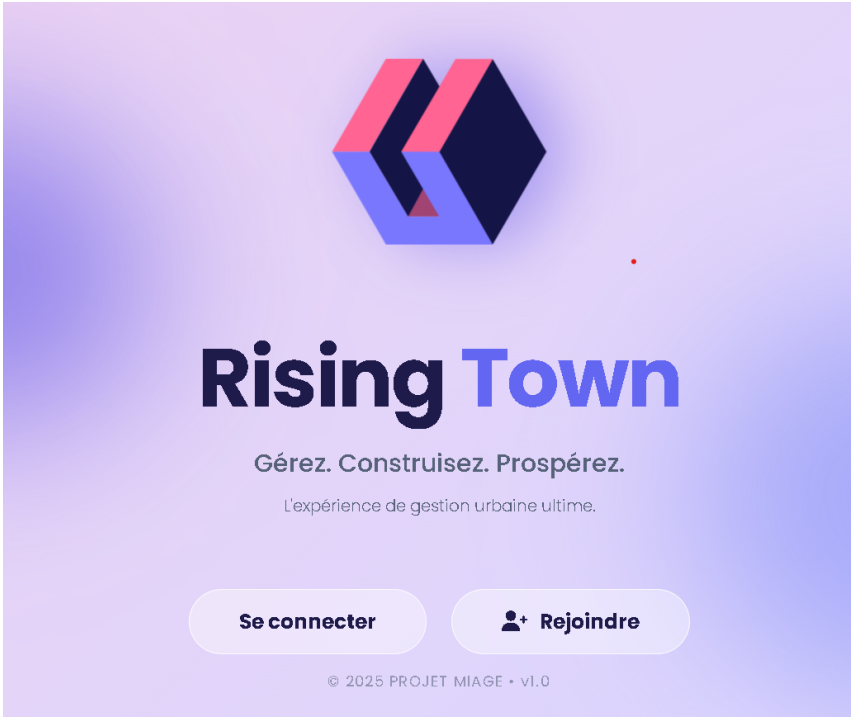


FIGURE 1.2 – Page d'accueil et point d'entrée dans la simulation

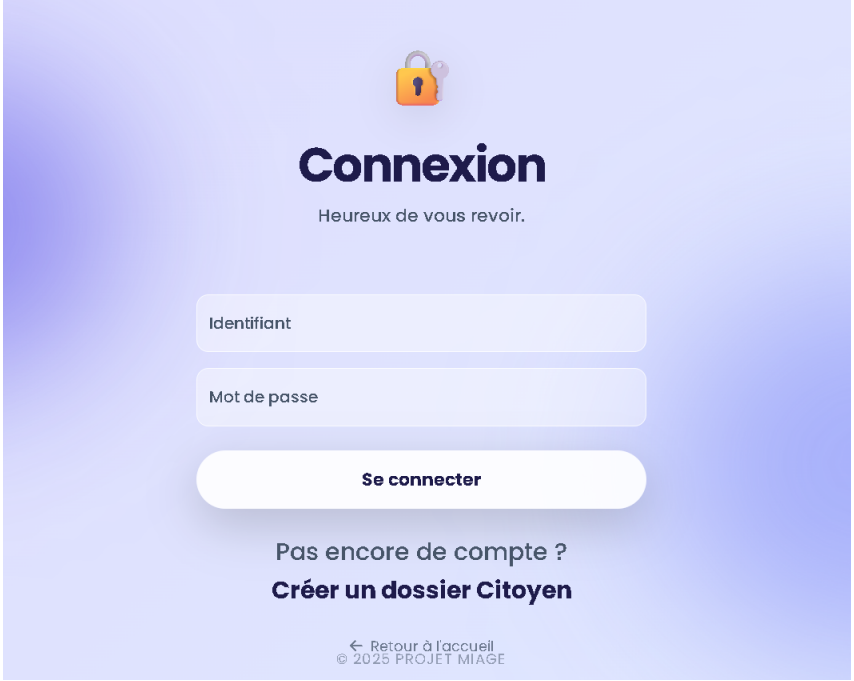


FIGURE 1.3 – Interface de connexion sécurisée (Login)

1.6.2 Espace Personnel du Citoyen

Une fois connecté, le citoyen accède à son tableau de bord (Dashboard). C'est le centre de contrôle qui affiche ses indicateurs vitaux (Budget, Santé, Bonheur) et la carte interactive.

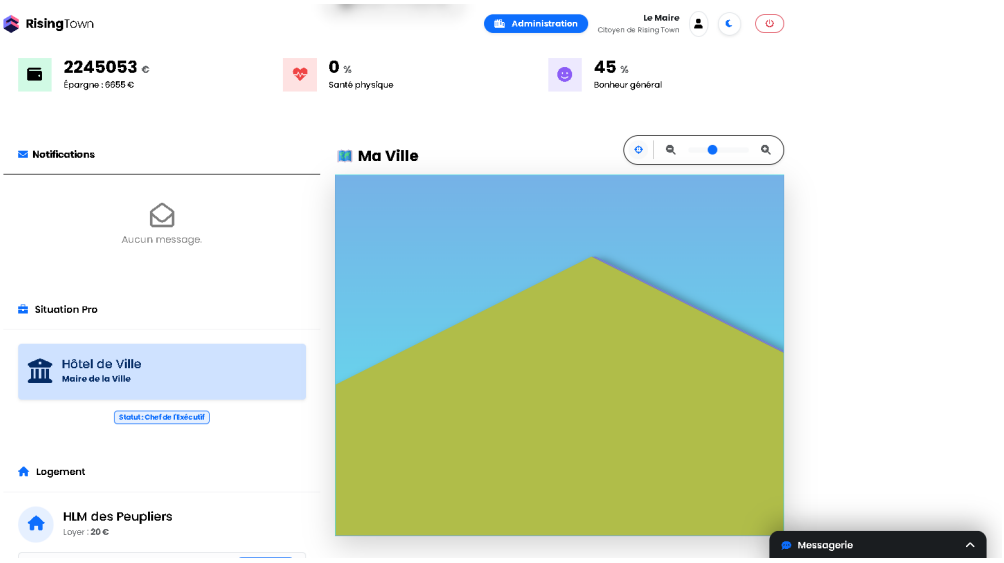


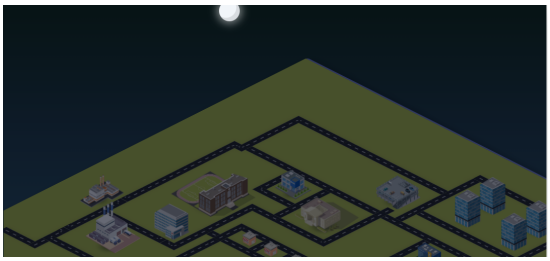
FIGURE 1.4 – Dashboard Citoyen : Vue globale des statistiques et de la ville

1.6.3 Immersion et Cycle Temporel

Le moteur du jeu gère un cycle jour/nuit dynamique qui modifie l'ambiance visuelle de la ville en temps réel, renforçant l'immersion du joueur.

(a) Journée standard

(b) Coucher de soleil



(c) Nuit

(d) Lever de soleil

FIGURE 1.5 – Évolution dynamique de l'environnement urbain selon l'heure

1.6.4 Outils de Communication et Justice

Le système intègre des fonctionnalités spécifiques pour l'interaction sociale et la gestion pénale.

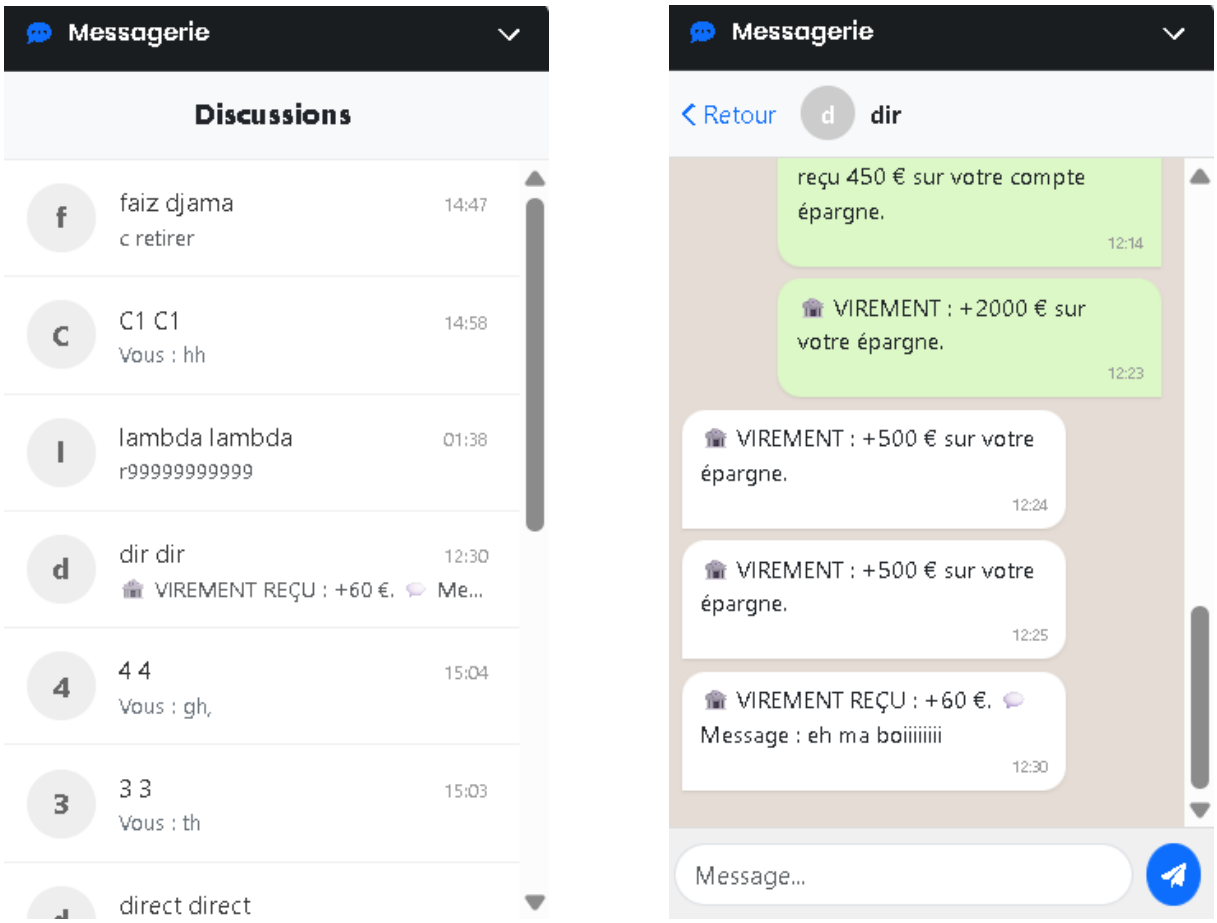


FIGURE 1.6 – Interface Smartphone : Liste des discussions (gauche) et Conversation active (droite)

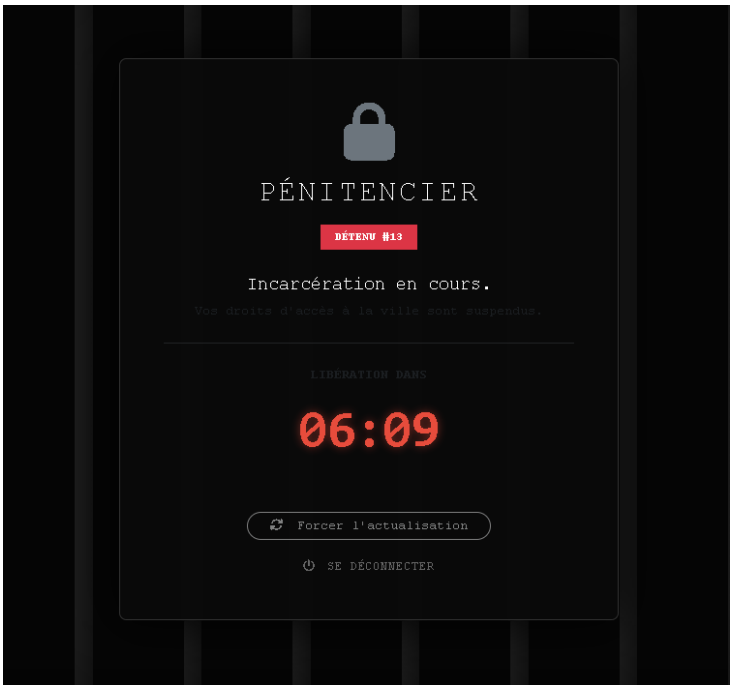


FIGURE 1.7 – Interface Pénitencier : Vue restrictive bloquant la navigation lors d'une incarcération

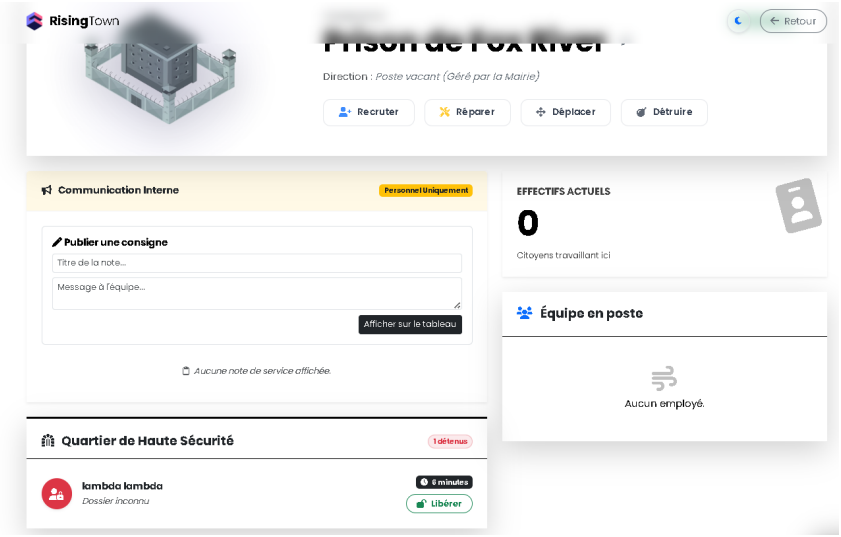


FIGURE 1.8 – Interface Administration : Registre des écrous et gestion des détenus

Chapitre 2

Analyse et Spécification des Besoins

2.1 Les Acteurs du Système

Le système gère quatre niveaux d'accréditation hiérarchiques.

Le Citoyen (Utilisateur Standard) : Il possède un budget, une jauge de santé et de bonheur. Il peut postuler à un emploi, louer un bien, acheter un véhicule et voter.

Le Directeur (Chef de service) : Nommé à la tête d'un bâtiment (Usine, Hôpital, Banque...). Il gère le recrutement, les licenciements et peut publier des notes de service.

L'Adjoint : Bras droit du Maire. Il possède les droits de maintenance (réparation des bâtiments) et de communication officielle (Actualités de la ville).

Le Maire (Super-Administrateur) : Il a tous les pouvoirs : construction, destruction, fiscalité, justice (prison) et nominations.

2.2 Formalisation des Besoins

Cette section détaille les exigences impératives auxquelles la solution *RisingTown* doit répondre pour être validée.

2.2.1 Besoins Fonctionnels (Ce que le système doit faire)

Les besoins fonctionnels décrivent les actions et comportements attendus du système. Ils sont classés par module.

F1 - Gestion de la Simulation Urbaine (Moteur de Jeu)
Le système doit permettre l'affichage d'une carte interactive (Grille isométrique). Il doit gérer le placement et la persistance des coordonnées (x, y) de chaque bâtiment, ainsi que la détection de collisions lors de la construction.

F2 - Gestion Économique et Temporelle
Le système doit automatiser les flux financiers sans intervention humaine constante. Cela inclut le prélèvement des loyers et le versement des salaires à une fréquence définie (action "Fin de Journée"). Le système doit empêcher les soldes négatifs non autorisés.

F3 - Gestion des Rôles et Permissions
Le système doit distinguer strictement les droits d'accès. Un Citoyen ne doit jamais pouvoir accéder à l'interface d'administration (Mairie), et un utilisateur non connecté ne doit avoir accès à aucune donnée sensible (Redirection vers Login).

F4 - Système de Communication et Notifications
Le système doit permettre l'échange de messages textuels entre utilisateurs. Il doit également générer des notifications système (Candidatures, Virements) qui persistent jusqu'à leur suppression par l'utilisateur.

F5 - Mécaniques de Vie (Santé/Bonheur)
Le système doit implémenter des jauges évolutives. La santé doit diminuer naturellement (fatigue) et remonter via des interactions spécifiques (Hôpital). Le bonheur doit influencer le gameplay (Risque d'émeutes).

2.2.2 Besoins Non Fonctionnels (Qualité de service)

Les besoins non fonctionnels définissent les contraintes techniques, ergonomiques et de qualité.

NF1 - Ergonomie et Utilisabilité (UI/UX)
L'interface doit être intuitive et ludique ("Gamification"). L'utilisateur doit comprendre son état (Budget, Santé) en un coup d'œil via un Tableau de Bord (Dashboard) clair. Le design doit être réactif (Responsive) pour s'adapter aux écrans standards.

NF2 - Sécurité des Données

- **Authentification :** Les mots de passe doivent être hachés (PBKDF2 via Django).
- **Protection CSRF :** Tous les formulaires doivent être protégés contre les requêtes falsifiées.
- **Isolation :** Un utilisateur ne doit pas pouvoir modifier les données d'un autre (ex : vider le compte d'un tiers) via manipulation d'URL.

NF3 - Performance et Robustesse
Le chargement de la carte de la ville doit se faire en moins de 2 secondes, même avec 50 bâtiments affichés. Le système doit gérer les accès concurrents (ex : deux citoyens postulant au même emploi simultanément) sans corruption de données.

NF4 - Maintenabilité et Évolutivité
Le code doit respecter l'architecture MVT de Django. Le couplage entre les modules (ex : Ville et Club de Foot) doit être faible pour permettre des mises à jour indépendantes.

NF5 - Disponibilité
L'application étant hébergée sur le cloud (PythonAnywhere), elle doit être accessible 24h/24 via une URL publique sécurisée (HTTPS).

2.3 Modélisation des Interactions (Use Case)

Le diagramme ci-dessous synthétise l'ensemble des fonctionnalités offertes par la plateforme.

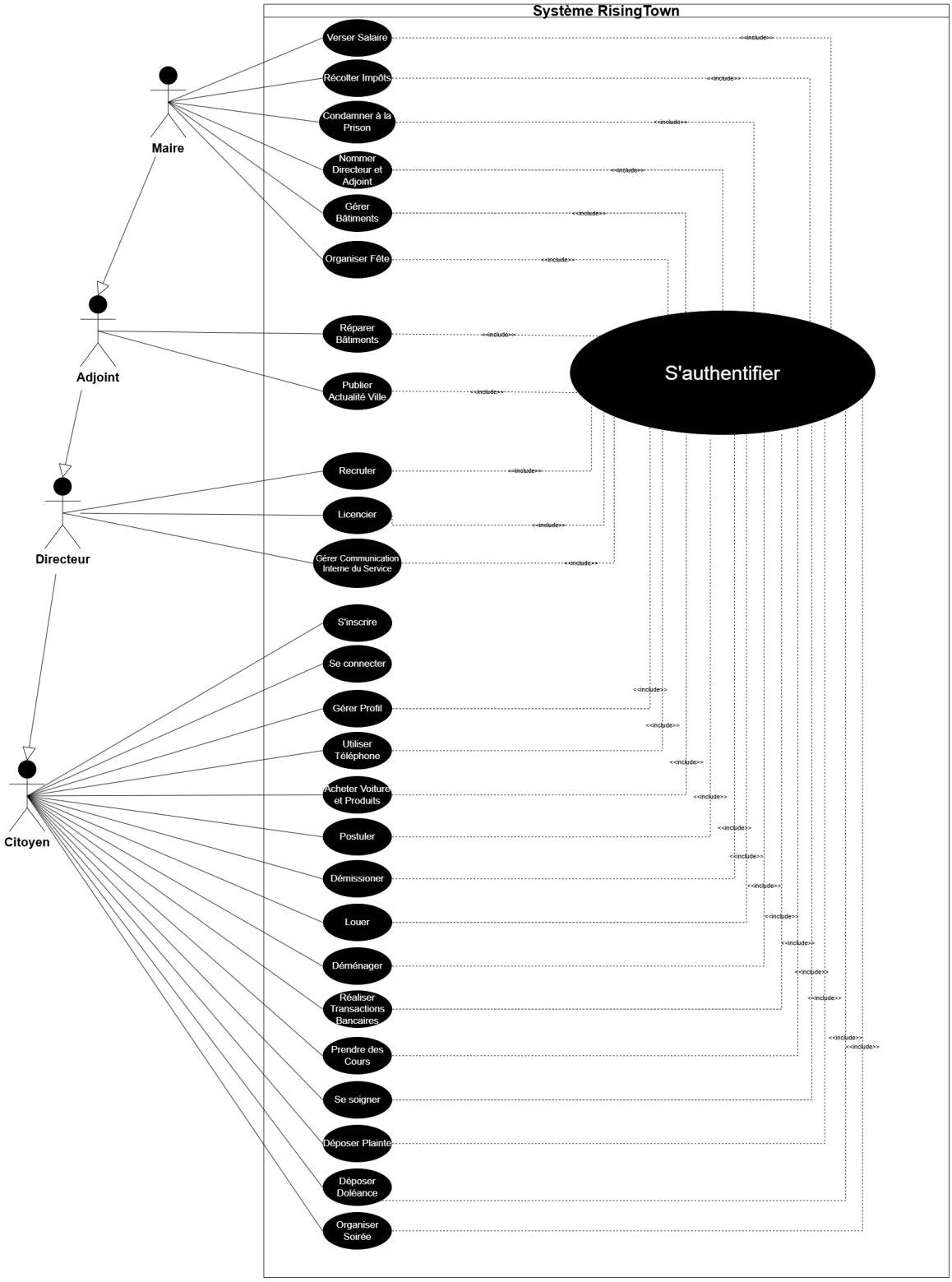


FIGURE 2.1 – Diagramme de Cas d'Utilisation Global - RisingTown

2.4 Règles de Gestion (Business Rules)

Le "moteur" du jeu repose sur des règles mathématiques et logiques strictes implémentées dans le Back-End Django.

2.4.1 Algorithme Fiscal (Impôts Progressifs)

Pour assurer l'équité sociale, le système (`collecter_impots`) applique un barème par tranches :

— **Pauvreté (< 100€) :** 0% d'impôt.

— **Classe Moyenne (100€ - 2000€) :** 5% à 10% d'impôt.

— **Richesse (> 2000€) :** 20% d'impôt.

Note : Les citoyens sans domicile fixe (SDF) sont exonérés d'impôts.

2.4.2 Système Judiciaire et Carcéral

La gestion de la prison est assurée par un **Middleware** (`PrisonMiddleware.py`) qui intercepte chaque requête HTTP.

1. Si le citoyen a le statut `est_en_prison = True`.
2. Le système compare `timezone.now()` avec `date_liberation`.
3. **Cas A :** La date est passée → Libération automatique + Notification.
4. **Cas B :** La peine court toujours → Redirection forcée vers la vue `cellule_prison`. Toutes les autres pages sont bloquées.

2.4.3 Cycle Économique (Salaires et Loyers)

La fonction de clôture de journée (`verser_salaires`) exécute les tâches suivantes en lot (Batch) :

1. Paiement des salaires aux employés (débit budget Ville → crédit Citoyen).
2. Versement du RSA aux chômeurs.
3. Prélèvement automatique des loyers (débit Citoyen → crédit Ville).
4. Envoi d'un bulletin récapitulatif via le système de notifications interne.

2.5 Spécifications Fonctionnelles par Rôle

Cette section détaille les actions possibles pour chaque acteur du système.

2.5.1 Espace Citoyen (Utilisateur Standard)

Le Citoyen est l'acteur central. Ses fonctionnalités couvrent la vie quotidienne.

Domaine	Fonctionnalités Détaillées

Authentification	- S'inscrire (Création compte User + Profil Citoyen). - Se connecter / Se déconnecter. - Modifier son profil (Email, Nom).
Logement	- Consulter la liste des habitations disponibles. - Emménager dans un logement (Paiement caution). - Payer son loyer (Prélèvement automatique). - Déménager (Quitter le logement).
Emploi	- Consulter les offres d'emploi par Bâtiment. - Postuler (Envoi de candidature). - Travailler (Actions spécifiques : Usine, Centrale...). - Démissionner.
Finances	- Consulter son solde (Argent liquide). - Utiliser la Banque (Dépôt, Retrait sur compte Épargne). - Effectuer des virements vers d'autres citoyens.
Vie Sociale	- Utiliser le Téléphone (Chat AJAX en temps réel). - Consulter les Actualités de la ville. - Déposer une doléance à la Mairie ou une plainte au Commissariat.

2.5.2 Espace Directeur (Gestionnaire)

Le Directeur est un citoyen nommé à la tête d'une structure.

Domaine	Fonctionnalités Détaillées
Ressources Humaines	- Recevoir les notifications de candidature. - Accepter ou Refuser un candidat. - Licencier un employé (avec motif).
Communication	- Publier une "Note de Service" (Actualité interne visible par les employés uniquement).
Gestion Activité	- Lancer la production (Usine) pour générer du budget. - Gérer les stocks (ex : Combustible pour la Centrale). - Fixer certains paramètres locaux (ex : Tarif du Parking).

2.5.3 Espace Adjoint (Administration)

L'Adjoint assiste le Maire et possède des droits techniques.

Domaine	Fonctionnalités Détaillées
Maintenance	- Visualiser l'état d'usure des bâtiments sur la carte. - Effectuer les réparations (Débit du budget Ville).
Information	- Publier des Actualités Officielles visibles par tous. - Modérer les contenus (Suppression d'actualités obsolètes).

2.5.4 Espace Maire (Super-Administrateur)

Le Maire possède les pleins pouvoirs sur la ville.

Domaine	Fonctionnalités Détaillées
Urbanisme	- Construire de nouveaux bâtiments sur la grille (Map). - Détruire des infrastructures existantes. - Déplacer des bâtiments. - Redimensionner la carte de la ville.
Politique & RH	- Nommer ou Révoquer les Directeurs de bâtiments. - Promouvoir un citoyen au rang d'Adjoint.
Finance Publique	- Collecter les impôts (Algorithme progressif par tranches). - Verser les salaires et les aides sociales (RSA).
Justice	- Consulter les casiers judiciaires. - Prononcer des peines de prison (Incarcération immédiate). - Accorder une grâce (Libération anticipée).

2.6 Product Backlog (Liste des User Stories)

Conformément à la méthodologie Scrum, voici la liste exhaustive des fonctionnalités à développer, priorisées et réparties par Sprints.

ID	Module	En tant que... Je veux...	Priorité	Sprint
US-01	Auth	Me connecter et m'inscrire en tant que Citoyen.	Haute	1
US-02	Ville	Voir la carte de la ville et les bâtiments existants.	Haute	1
US-03	Profil	Modifier mes informations personnelles (Email, Nom).	Moy.	1
US-04	Logement	Louer un appartement libre et payer la caution.	Haute	2
US-05	Emploi	Voir la liste des chômeurs (Directeur) et Recruter.	Haute	2
US-06	Emploi	Postuler à une offre d'emploi (Citoyen).	Haute	2
US-07	Finance	Recevoir mon salaire ou mon RSA automatiquement à minuit.	Haute	2
US-08	Mairie	Construire un nouveau bâtiment sur la grille (x,y).	Haute	3
US-09	Mairie	Collecter les impôts selon la richesse des citoyens.	Moy.	3
US-10	Police	Incarcérer un citoyen (Bloquer sa navigation).	Basse	3
US-11	Justice	Consulter le casier judiciaire d'un suspect.	Basse	3
US-12	Social	Envoyer un SMS (Chat) à un autre citoyen.	Moy.	4
US-13	Social	Recevoir une notification (AJAX) en cas de nouveau message.	Moy.	4

ID	Module	En tant que... Je veux...	Priorité	Sprint
US-14	Santé	Me faire soigner à l'hôpital contre de l'argent.	Basse	4
US-15	Social	Organiser une fête (Maire) ou une soirée (Citoyen) pour le moral.	Moy.	4
US-16	Évènement	Subir des émeutes si le bonheur global chute sous 30%.	Haute	4

Chapitre 3

Conception Technique

3.1 Environnement de Développement

Le projet est conçu pour être déployé dans un environnement standard Windows.

Composant	Spécification
OS	Windows 10 / 11 (Architecture 64 bits)
Langage	Python 3.9+
Framework Web	Django 5.2.8
SGBD	SQLite3 (Fichier local `db.sqlite3`)
Serveur Web	WSGI (Développement via `manage.py`)
Front-End	HTML5, CSS3, JavaScript (AJAX, Fetch API)

3.2 Justification des Choix Techniques

3.2.1 Comparatif Framework Backend : Django vs Flask

Pour le développement du serveur, une étude comparative a été menée.

Critère	Flask (Micro-framework)	Django (Full-stack)
Architecture	Minimaliste, nécessite beaucoup de bibliothèques tierces.	"Batteries included" (Auth, Admin, ORM intégrés).
Base de Données	Configuration manuelle (SQLAlchemy).	ORM natif puissant et migrations automatiques.
Sécurité	À implémenter manuellement (CSRF, XSS).	Protections natives activées par défaut.
Administration	À créer de zéro.	Interface d'administration générée automatiquement.

Décision : Le choix s'est porté sur **Django** pour sa robustesse, sa gestion native des utilisateurs (`AbstractUser`) et sa rapidité de mise en place grâce à l'interface d'administration, essentielle pour le rôle du Maire au début du projet.

3.2.2 Architecture MVT (Modèle-Vue-Template)

Contrairement au MVC classique, Django utilise le MVT.

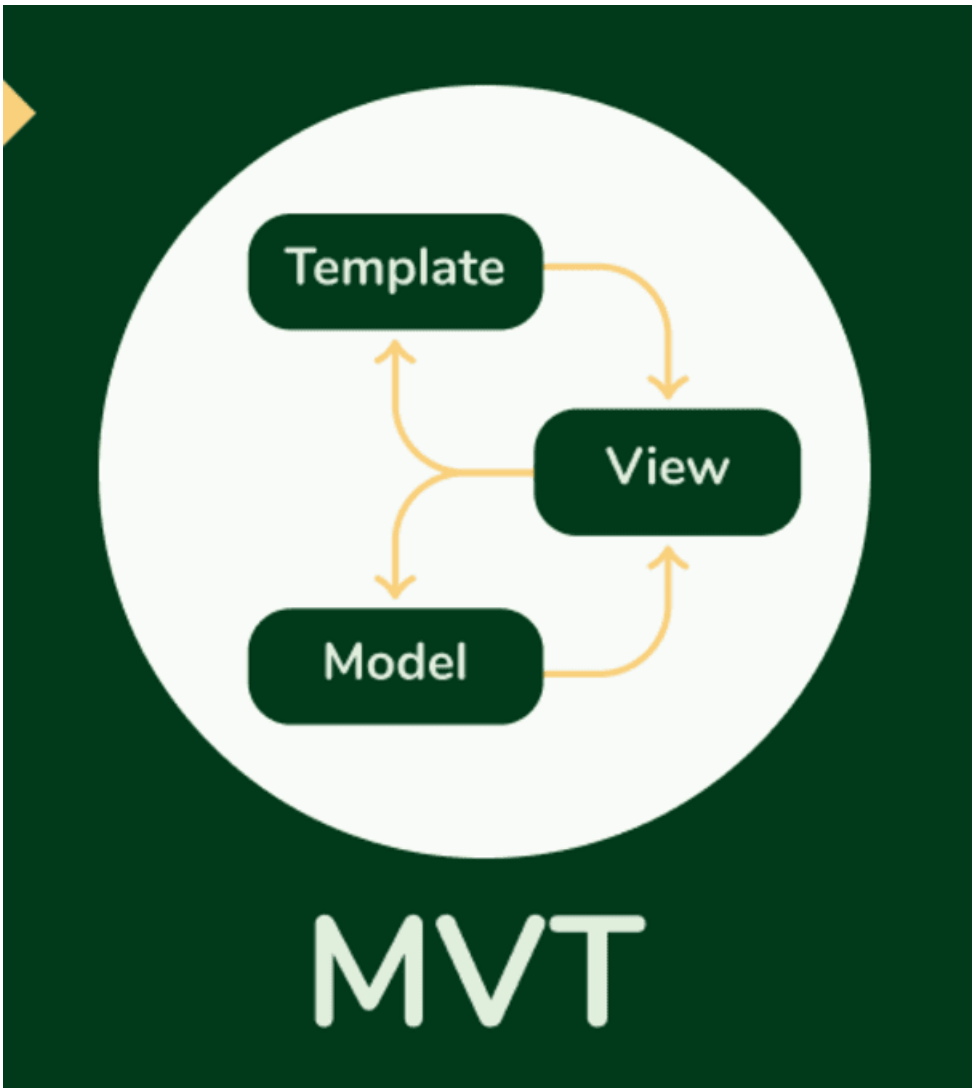


FIGURE 3.1 – Architecture MVT du projet RisingTown

3.3 Diagramme de Classes (Modèle de Données)

Le schéma de base de données est complexe, gérant les relations circulaires entre les Citoyens (qui habitent des bâtiments) et les Bâtiments (qui emploient des citoyens).

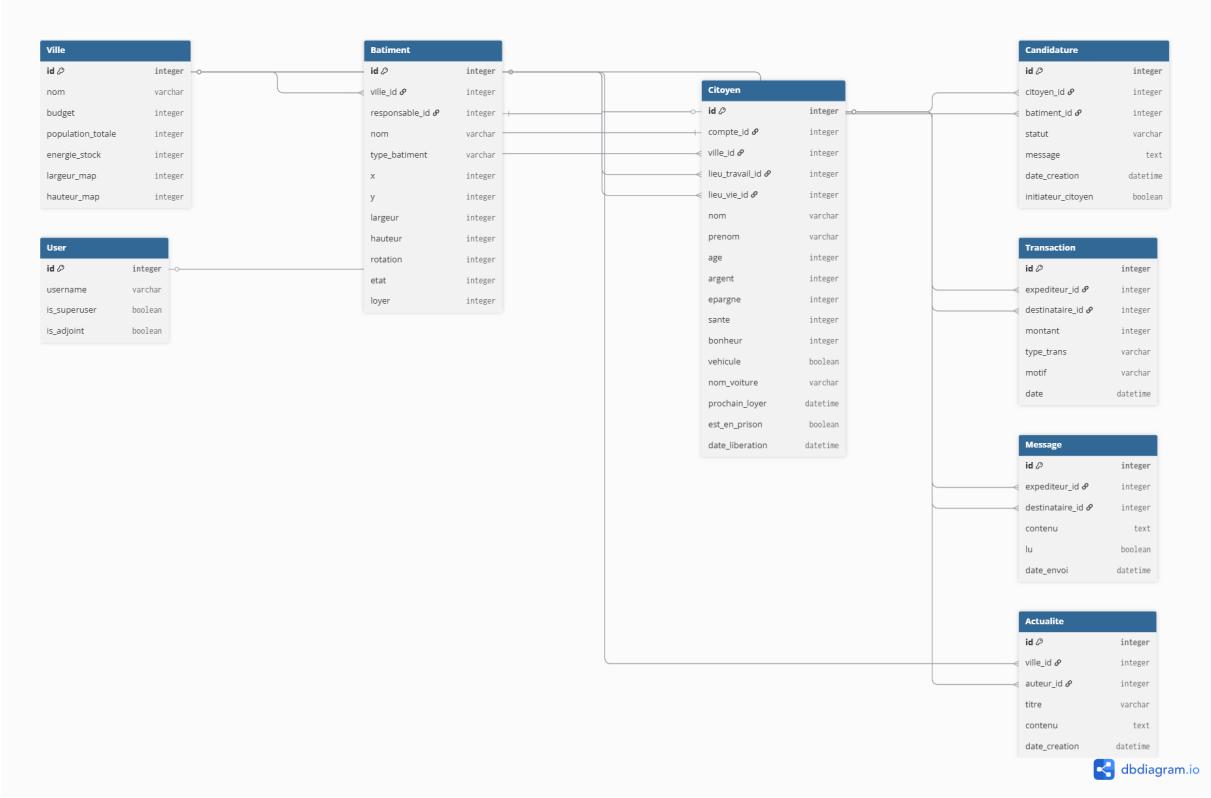


FIGURE 3.2 – Diagramme de Classes Complet (Généré depuis models.py)

3.4 Dictionnaire des Données

Cette section détaille la structure exacte de la base de données, telle que définie dans les fichiers `models.py`.

3.4.1 Table : Citoyen (Le Joueur)

Attribut	Type	Description
id	BigAutoField	Identifiant unique.
compte	OneToOne	Lien vers le système d'authentification Django (User).
nom / prenom	CharField	Identité civile.
argent	Integer	Argent liquide disponible.
epargne	Integer	Argent sécurisé en banque.
sante	Integer	Jauge de 0 à 100.
bonheur	Integer	Jauge de 0 à 100.
est_en_prison	Boolean	Si Vrai, bloque la navigation.
lieu_vie	ForeignKey	Lien vers le Bâtiment d'habitation.
lieu_travail	ForeignKey	Lien vers le Bâtiment employeur.
vehicule	Boolean	Possède une voiture personnelle.
nom_voiture	CharField	Marque/Modèle du véhicule.
date_liberation	DateTime	Date exacte de fin de peine.
prochain_loyer	DateTime	Date du prochain prélèvement.

3.4.2 Table : Batiment (Infrastructures)

Attribut	Type	Description
nom	CharField	Nom du bâtiment (ex : "Hôpital Central").
type_batiment	CharField	Énumération (MAIRIE, USINE, MAISON, ROUTE, ROUTE_VIRAGE, PARKING...).
x / y	Integer	Coordonnées sur la grille de la ville.
largeur / hauteur	Integer	Dimensions occupées sur la carte.
cout_construction	Integer	Prix pour le budget de la ville.
etat	Integer	Pourcentage d'usure (nécessite réparation).
responsable	ForeignKey	Le Citoyen nommé Directeur.
rotation	Integer	Angle d'orientation des routes (0, 90, 180, 270).
loyer	Integer	Montant du loyer (si habitation).

3.4.3 Table : Candidature (Système de Notifications)

Cette table est centrale : elle gère tout l'historique des interactions (Offres d'emploi, Plaintes, Virements, Doléances).

Attribut	Type	Description

statut	CharField	EN_ATTENTE, ACCEPTEE, REFUSEE, PLAINTE, DOLEANCE.
message	TextField	Contenu textuel de la notification.
initiateur_citoyen	Boolean	Définit le sens de la demande (Citoyen vers Maire ou inverse).

3.4.4 Table : Transaction (Historique Bancaire)

Attribut	**Type**	**Description**
montant	Integer	Valeur de la transaction.
type_trans	CharField	DEPOT, RETRAIT, VIREMENT, SALAIRE.
expediteur	ForeignKey	Citoyen émetteur.
destinataire	ForeignKey	Citoyen bénéficiaire.
motif	CharField	Justificatif du virement.

3.5 Détail des Processus Critiques (Diagrammes de Séquence)

3.5.1 Analyse Approfondie du Processus de Clôture de Journée

Le diagramme de séquence ci-dessous illustre le processus métier le plus complexe de l'application *RisingTown* : la distribution des revenus et la gestion des loyers. Ce processus est critique car il modifie massivement l'état de la base de données (entités `Citoyen`, `Ville`, `Candidature`) en une seule transaction logique.

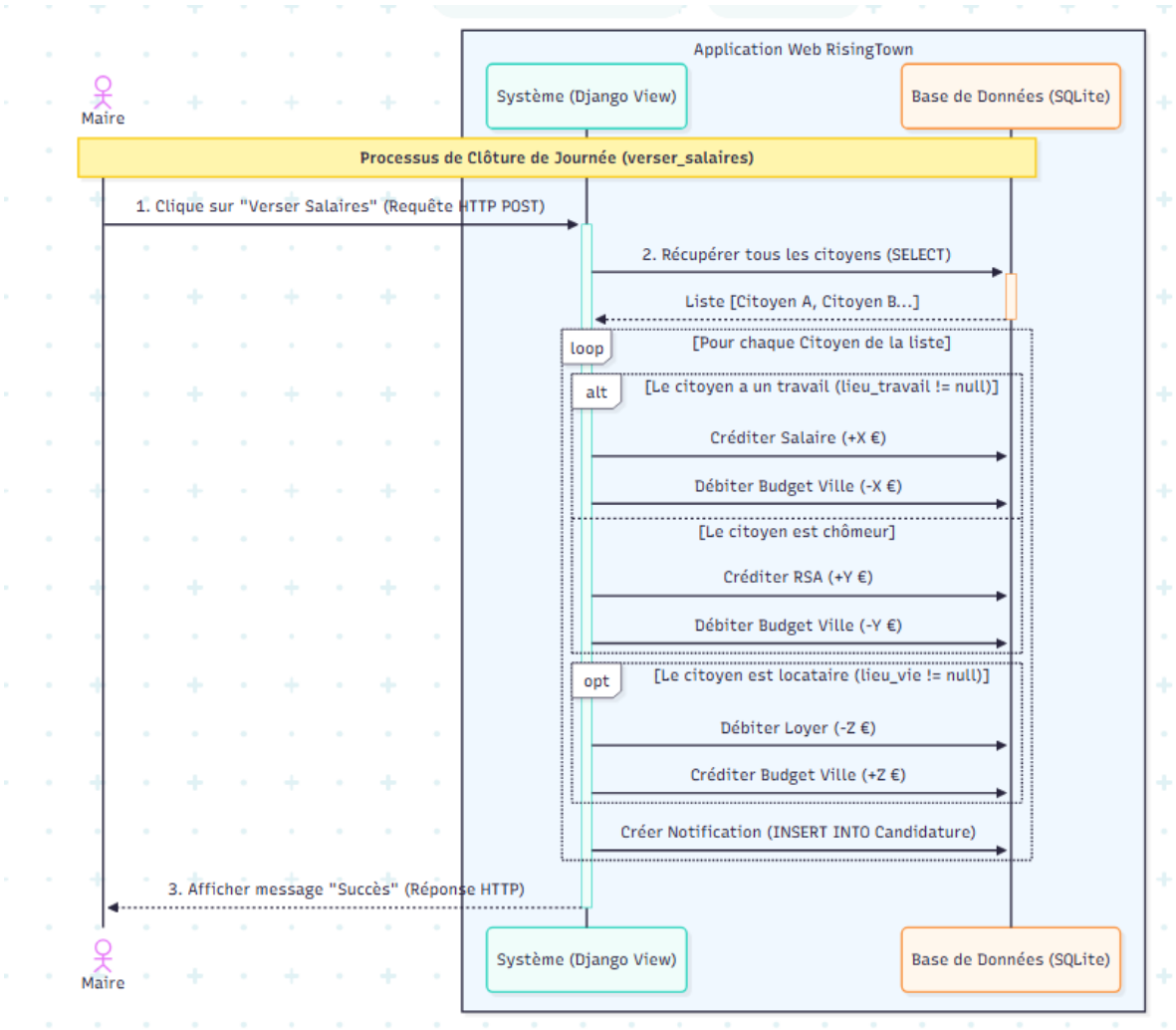


FIGURE 3.3 – Diagramme de Séquence : Processus de Clôture de Journée (verser_salaires)

Description Séquentielle des Flux

Le processus se déroule en trois phases distinctes, impliquant l'acteur **Maire**, le **Système** (la vue Django) et la **Base de Données** (SQLite).

Phase 1 : Déclenchement et Récupération des Données

— **Étape 1 (Requête Client) :** L'acteur *Maire* déclenche l'action via l'interface utilisateur en cliquant sur le bouton "Verser Salaires". Cela génère une requête `HTTP POST` vers le serveur.

— **Étape 2 (Requête SGBD) :** Le Système reçoit la demande et interroge la Base de Données pour récupérer l'intégralité de la population. Une requête `SELECT` est exécutée pour charger en mémoire la liste des objets `Citoyen` ainsi que leurs relations (travail, logement).

— **Retour :** La Base de Données renvoie une liste itérable d'objets (Citoyen A, Citoyen B, etc.).

Phase 2 : Traitement Itératif (Boucle Loop) Le cœur du processus est une boucle (`loop`) qui parcourt chaque citoyen de la liste récupérée. Pour chaque itération, le système applique des règles de gestion strictes conditionnées par des blocs alternatifs (`alt`) et optionnels (`opt`).

1. **Bloc Alternatif (Alt) - Gestion des Revenus :** Le système vérifie le statut professionnel du citoyen via l'attribut `lieu_travail`.
   — **Cas Nominal (Travailleur) :** Si le citoyen a un emploi (`lieu_travail != null`), le système effectue deux mises à jour (`UPDATE`) : il crédite le compte du citoyen du montant de son salaire (+X €) et débite le budget de la Ville du même montant.
   — **Cas Alternatif (Chômeur) :** Si le citoyen est sans emploi, le système applique le filet de sécurité social. Il crédite une allocation RSA (+Y €) au citoyen et débite le budget de la Ville.
2. **Bloc Optionnel (Opt) - Gestion du Logement :** Le système vérifie si le citoyen est locataire via l'attribut `lieu_vie`.
   — Si `lieu_vie != null`, un prélèvement automatique est déclenché. Le système débite le loyer (-Z €) du compte du citoyen et crédite le budget de la Ville (ou du propriétaire) via des requêtes `UPDATE`.
3. **Traçabilité (Notification) :** Avant de passer à l'itération suivante, le système génère une notification récapitulative. Une requête `INSERT INTO` est envoyée pour créer un nouvel objet dans la table `Candidature` (utilisée ici comme système de messagerie générique), contenant le bilan financier de la journée pour ce citoyen spécifique.

Phase 3 : Clôture et Réponse Une fois la boucle terminée (tous les citoyens ont été traités), le Système renvoie une réponse `HTTP` au Maire (Étape 3). L'interface affiche alors un message de succès ("Journée terminée"), confirmant que tous les mouvements financiers ont été enregistrés avec succès.

Chapitre 4

Organisation et Livrables

4.1 Livrables Fournis

Conformément aux attentes, le rendu du projet comprend les éléments suivants :

1. **Code Source Complet :**
   - Le dossier `myapp/` contenant toute la logique (views, models, forms).
   - Le dossier `templates/` avec les interfaces HTML/CSS.
   - Les fichiers de configuration (`settings.py`, `urls.py`).
2. **Base de Données Initialisée :**
   - Fichier `db.sqlite3` contenant une ville pré-configurée ("RisingTown") avec une Mairie, quelques routes et les premiers citoyens.
3. **Documentation Technique :**
   - Ce Cahier des Charges.
   - Un fichier `README.md` expliquant la procédure d'installation (création de l'environnement virtuel, installation des requirements, lancement du serveur).

4.2 Conclusion et Perspectives

Le projet *RisingTown* réussit à modéliser la complexité d'une ville moderne au travers d'une interface web accessible. L'architecture modulaire basée sur Django permet d'envisager de futures évolutions, telles que :

- L'ajout d'un mode multijoueur massif avec WebSockets (Django Channels).
- Une gestion plus fine de l'énergie (Réseau électrique complexe).
- Une application mobile native utilisant les mêmes API REST que celles développées pour le "Téléphone virtuel".
- La mise en place d'un système de **SSO (Single Sign-On)** pour permettre aux citoyens de se connecter au Club de Foot avec leurs identifiants RisingTown.